AI-Powered HR Resume Screening & Ranking System (SmartFilter for HR)

Team Name & Members:

SHARAN R H - 2024UG000313

GNANENDRA REDDY A - 2024UG000329

VENKATESH K - 2024UG000335

PAVAN KALYAN M - 2024UG000202

PRJAWAL M A - 2024UG000240

MALLESHA S - 2024UG000275

Institution/Organization Name:

Vidyashilp University

Course:

Artificial Intelligence

Date of Submission:

31/03/2026

TABLE OF CONTENTS

1. Introduction
2. Problem Statement
3. Objectives
4. Solution Overview
5. Technical Stack Used
6. System Architecture
7. Implementation Details
8. LLM and AI Integration
9. Frontend & UI Design
10. Code Structure & Execution Guide
11. Results & Output
12. Demo Video Link
13. Individual Contributions
14. Impact of the Solution
15. Future Enhancements
16. Conclusion
17. References & Citations

1. INTRODUCTION

This project presents an AI-powered Applicant Tracking System (ATS) designed to streamline and automate the recruitment process. The system is capable of automatically ingesting resumes via email, parsing them using advanced Large Language Models (LLMs), and ranking candidates based on job descriptions using semantic similarity techniques.

Traditional recruitment processes are manual, time-consuming, and inefficient. HR professionals often spend hours reviewing resumes, which delays hiring and increases the risk of missing qualified candidates. This system addresses these challenges by introducing automation and intelligence into the hiring workflow.

2. PROBLEM STATEMENT

Recruitment systems today face significant challenges due to the unstructured nature of resume data. Resumes are submitted in various formats such as PDF and DOCX, making it difficult to extract relevant information using traditional parsing techniques.

Additionally, most systems rely on keyword matching, which fails to capture the true semantic meaning of candidate skills and experience. This results in inaccurate candidate ranking and inefficient hiring decisions.

This project aims to solve these issues by developing an AI-based system that extracts structured data and performs semantic matching between resumes and job descriptions.

3. OBJECTIVES

The primary objectives of this project are:

- To automate resume collection using an email-based system
- To extract structured data such as name, skills, and experience using AI
- To store candidate information in a database
- To allow HR users to create job descriptions
- To rank candidates based on semantic similarity
- To provide a user-friendly dashboard for HR operations
-

4. SOLUTION OVERVIEW

The system follows a modular and decoupled architecture.

In the first phase, resumes are collected either through email ingestion or manual upload. These resumes are processed using AI to extract structured information, which is then stored in the database.

In the second phase, HR users input job descriptions into the system. The AI engine compares job descriptions with stored resumes and generates a match score. Candidates are then ranked based on this score and displayed in the dashboard.

5. TECHNICAL STACK USED

Programming Languages:

Python (Backend), HTML5, CSS3, JavaScript (Frontend)

Frameworks and Libraries:

FastAPI, Uvicorn, Tailwind CSS, Alpine.js

Databases and APIs:

SQLite (Database), Pinecone (Vector Database), Google Gemini AI

Other Tools:

WebSockets, Email Integration (IMAP), Resume Parsing Libraries

6. SYSTEM ARCHITECTURE

The system consists of four main components:

1. Frontend Interface – Provides an interactive dashboard for HR users
2. Backend API – Handles data processing and communication
3. AI Processing Layer – Performs resume parsing and ranking
4. Database – Stores structured candidate and job data

The workflow is as follows:

Resumes → AI Parsing → Database Storage → Job Input → AI Matching → Candidate Ranking → Dashboard Display

7. IMPLEMENTATION DETAILS

The backend is developed using FastAPI, which provides RESTful APIs for managing candidates, jobs, and rankings.

The frontend is designed using HTML, Tailwind CSS, and JavaScript, providing a modern and responsive dashboard.

The database is implemented using SQLite, with multiple tables such as candidates, jobs, rankings, and notifications.

Advanced features such as WebSockets are used for real-time updates, and a background worker is implemented for automatic email ingestion.

6. LLM AND AI INTEGRATION

The system integrates AI in multiple stages:

Resume Parsing:

Google Gemini AI is used to extract structured data from unstructured resumes.

Semantic Search:

Text embeddings are generated for both resumes and job descriptions.

Ranking Engine:

Candidates are ranked using cosine similarity between embeddings, producing a match score.

9. FRONTEND & UI DESIGN

The user interface is designed to be clean, professional, and user-friendly. It includes:

- Dashboard with statistics
- Candidate listing and filtering
- Job creation interface
- Ranking system with match percentage
- Notification system

The UI follows a modern enterprise-style design, ensuring ease of use and clarity.

1 0. CODE STRUCTURE & EXECUTION GUIDE

Project Structure:

backend/

frontend/

database/

main.py

requirements.txt

Execution Steps:

1. Install dependencies using pip install -r requirements.txt
2. Configure environment variables in .env file
3. Run the server using python main.py
4. Open the application in browser

11. RESULTS & OUTPUT

The system successfully performs the following tasks:

- Resume parsing with high accuracy
- Storage of structured candidate data
- Job creation and management
- AI-based candidate ranking
- Real-time notifications

The system reduces manual effort and improves efficiency in recruitment.

12. DEMO VIDEO LINK

[Google Drive= https://drive.google.com/drive/folders/1Mu1RH0L7NxrQkrDIMHCbYY0FUKAckUyX?usp=sharing]

13. INDIVIDUAL CONTRIBUTIONS

1. **SHARAN R H (2024UG000313):** Backend Development (Co-developed the FastAPI architecture, API endpoints, and core Python processing logic). [GitHub Link= https://github.com/sharan-MAC/Recruit.AI---Intelligent-Recruitment-Dashboard]
2. **MALLESHA S (2024UG000275):** Backend Development (Co-developed the FastAPI architecture, and integrated Gemini AI for structured JSON parsing and vector generation). [GitHub Link= https://github.com/Mallesha-07/HR-SmartFilter.git]
3. **GNANENDRA REDDY A (2024UG000329):** Database Setup, Resume Collection & Reports (Designed the Supabase relational schemas, configured Pinecone vector indexing, and co-developed the IMAP email extraction pipeline). [GitHub Link= https://github.com/greddy9499/SmartFilter-HR-AI]
4. **PAVAN KALYAN M (2024UG000202):** Database Setup, Resume Collection & Reports (Managed Supabase authentication setup, Pinecone integration, and the automated resume ingestion workflow). [GitHub Link= https://github.com/pavankalyan86908-lab/SmartFilter-HR-AI]
5. **VENKATESH K (2024UG000335):** Frontend Development & Presentation PPT (Co-developed the Vanilla JS user interface, implemented API fetch calls, and co-authored the final project PPT and Report). [Insert GitHub Link]

6. **PRJAWAL M A (2024UG000240):** Frontend Development & Presentation PPT (Designed the HTML/CSS layout, modal UI/UX interactions, and co-authored the final project PPT and Report). [GitHub Link= https://github.com/prajwalma4/Smart-Filter]

14. IMPACT OF THE SOLUTION

This system benefits HR professionals by reducing manual workload and improving hiring efficiency.
It enables faster decision-making, reduces bias, and ensures better candidate selection based on actual skills rather than keyword matching.

15. FUTURE ENHANCEMENTS

- Integration with job portals
- Interview scheduling automation
- Bias detection in hiring
- Advanced machine learning models
- Candidate feedback system

16. CONCLUSION

This project demonstrates how AI can transform traditional recruitment systems into intelligent, automated platforms. By combining modern web technologies with AI, the system provides a scalable and efficient solution for candidate screening and ranking.

17. REFERENCES & CITATIONS

- Google Generative AI Documentation
- Pinecone Vector Database Documentation
- FastAPI Documentation
- Tailwind CSS Documentation